Placement Portal System Report

1. Student Details

Name: Zaira Izhar

Roll Number: 23f3001132

Email: 23f3001132@ds.study.iitm.ac.in

About Me

I am pursuing the IIT Madras BS Degree in Data Science. I have a strong interest in Full Stack Web Development, Data Science, Artificial Intelligence, and Backend Development. I enjoy building scalable web applications using modern technologies and continuously improving my software development skills through practical projects.

2. Project Details

Project Title

Placement Portal System

Problem Statement

To design and develop a web-based Placement Portal that automates the recruitment process for educational institutions by connecting Students, Companies, and Placement Administrators on a single platform.

The system eliminates manual paperwork by managing company registrations, placement drives, student applications, interview scheduling, report generation, and automated notifications.

Approach

The application was developed using **Flask** as the backend framework and **Vue.js** as the frontend framework following a modular architecture.

The system provides secure role-based authentication using JWT and supports three different user roles:

- Student
- Company
- Administrator

Redis caching and Celery background jobs were integrated to improve performance and automate repetitive tasks such as email reminders, CSV exports, and monthly reports.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) in a limited way (~5-8%) mainly for wording improvements, small

refactoring suggestions, and documentation formatting. Core implementation decisions, route integration, debugging, testing, and final validation were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Backend Framework
SQLAlchemy	ORM for SQLite Database
SQLite	Database
Flask-JWT-Extended	JWT Authentication
Flask-Caching	Redis Caching
Redis	Caching & Message Broker
Celery	Background Jobs
Flask-Mail	Email Notifications
ReportLab	PDF Generation
Vue.js 3	Frontend Framework
Vue Router	Client-side Routing
Axios	REST API Communication
Bootstrap 5	Responsive UI Design
HTML5	Frontend Structure
CSS3	Styling

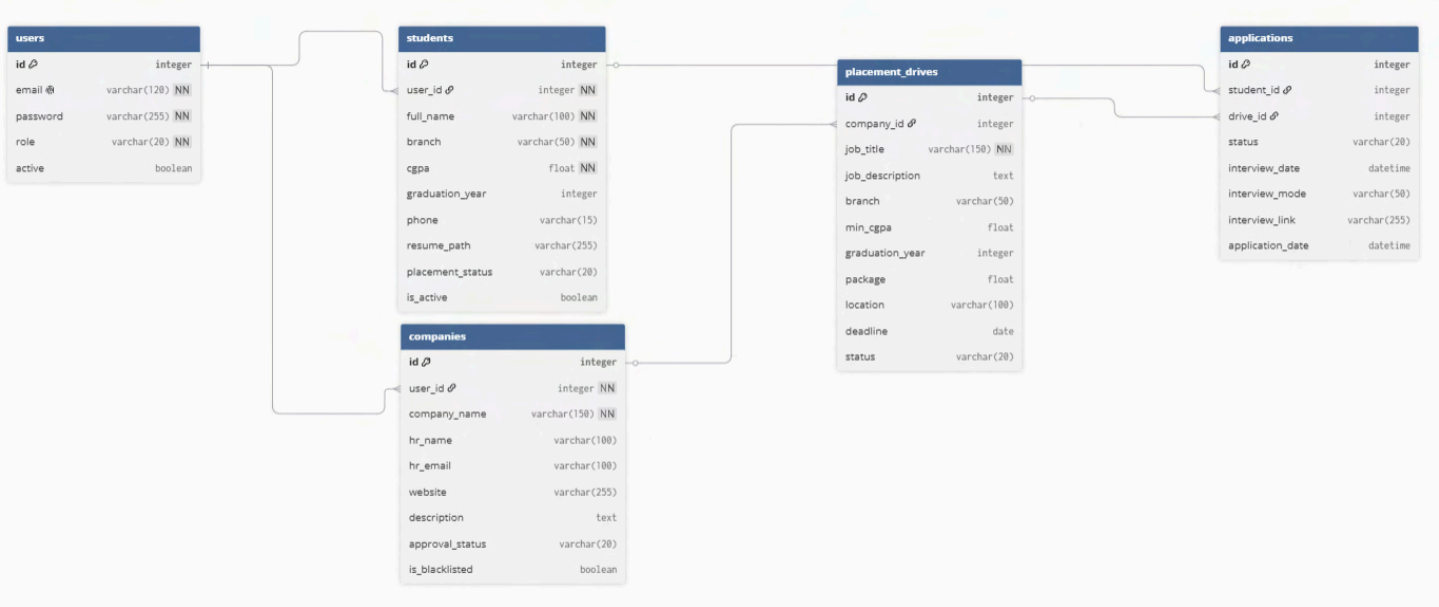
5. Database Schema / ER Summary

Main Tables

User, Student, Company, PlacementDrive, Application

Key Relationships

- User → Student: One-to-One (Each student has one user account.)
- User → Company: One-to-One (Each company has one user account.)
- Company → PlacementDrive: One-to-Many (A company can create multiple placement drives.)
- Student → Application: One-to-Many (A student can apply for multiple placement drives.)
- PlacementDrive → Application: One-to-Many (Each placement drive can receive multiple student applications.)
- Company → Application (Indirect): Companies can manage applications received for their placement drives through the PlacementDrive relationship.



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	User Login
/register/student	POST	Student Registration
/register/company	POST	Company Registration
/student/drives	GET	View Approved Drives
/student/apply/{id}	POST	Apply for Drive
/student/profile	GET/PUT	View & Update Profile
/company/create-drive	POST	Create Placement Drive
/company/my-drives	GET	View Company Drives
/company/applicants/{id}	GET	View Applicants
/admin/dashboard	GET	Admin Dashboard
/admin/analytics	GET	Analytics
/admin/company/{id}/approve	PUT	Approve Company
/admin/drive/{id}/approve	PUT	Approve Drive
/admin/report/pdf	GET	Generate PDF Report
/admin/report/html	GET	Generate HTML Report

YAML API Definition

Included separately as **api.yaml**.

7. Architecture and Core Features

Architecture

- **Application Entry:** app.py initializes the Flask application, SQLAlchemy database, JWT Authentication, Redis Cache, Flask-Mail, and Celery background workers.
- **Backend:** Database models are defined in models.py, REST APIs are implemented using Flask Blueprints in the routes/ directory, asynchronous tasks are managed in tasks.py, and Celery configuration is handled in celery_app.py.
- **Frontend:** Built using **Vue.js 3** with reusable components, Vue Router for navigation, Axios for backend communication, and Bootstrap 5 for responsive UI design.
- **Database:** SQLite is used for persistent storage with SQLAlchemy ORM managing database operations.
- **Background Processing:** Redis and Celery are integrated to execute asynchronous tasks such as email reminders, CSV export, and monthly report generation.

Core Features

- **Administrator:** Approve or reject company registrations, approve placement drives, manage students and companies, search records, blacklist companies, deactivate students, monitor analytics, and generate PDF/HTML reports.
- **Company:** Register company profile, create placement drives, manage drives, view applicants, schedule interviews, update application status, and generate offer letter PDFs.
- **Student:** Register and login securely, view approved placement drives, perform live search, apply for drives, upload resume, use ATS Resume Checker, manage profile, and export application history as CSV.
- **Authentication & Security:** Secure JWT-based authentication with role-based authorization for Students, Companies, and Administrators.
- **Automation:** Daily placement deadline reminder emails, monthly admin report emails, asynchronous CSV export, Redis caching, and automated report generation using Celery background jobs.

8. Backend Jobs and Caching

Scheduled Job – Daily Placement Reminder

Automatically sends reminder emails to students whose applied placement drives are approaching their application deadline. The job runs periodically using Celery Beat and sends HTML email notifications through Flask-Mail.

Scheduled Job – Monthly Admin Report

Generates a monthly placement summary containing statistics such as approved companies, placement drives, student applications, and placements. The report is generated automatically and delivered to the administrator via HTML email.

User Triggered Async Job – CSV Export

Students can request an export of their application history. The CSV file is generated asynchronously using Celery, and the completed file is emailed to the student as an attachment. The application supports task polling to monitor export status.

Caching

Redis caching is implemented to improve application performance by reducing repeated database queries. Frequently accessed APIs such as placement drives, dashboard statistics, student search, company search, analytics, and approved drive listings are cached with timeout-based expiration.

Video link

https://drive.google.com/file/d/1MQbYYrXsf7JXMuxzU4vWvOG2f3WywEoG/view?usp=drive_link